

Lab 01 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde stramien: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je zelf aan de terminal kunt controleren.

Vraag 1 — "Een container is een kleine VM"

Antwoord. Fout op het essentiële punt: een container bevat **geen besturingssysteem** en heeft **geen eigen kernel**. Het is een proces van de host, geïsoleerd door *namespaces*, begrensd door *cgroups* en uitgevoerd door de kernel van de host.

Waarom. Een VM start eerst een kernel, dan een `init`, dan tientallen systeemdiensten (logging, cron, SSH, netwerk...) — en pas daarna je applicatie. Vandaar de seconden of minuten opstarttijd en de gigabytes schijfruimte. Een container start niets van dat alles: de kernel draait al, en we vragen hem alleen om namespaces aan te maken en **één** proces te lanceren. De opstartkost is die van een `fork` + `exec`, enkele milliseconden; de schijfkost beperkt zich tot de bibliotheken die de applicatie echt nodig heeft.

→ LINUX

`fork` dupliceert het lopende proces, `exec` vervangt de inhoud ervan door een ander programma. Zo ontstaat *elk* Linux-proces, `podman run` inbegrepen: Podman dupliceert zichzelf, de kloon stapt in zijn namespaces en voert daarna jouw commando uit. Een container komt precies zo ter wereld als een `ls`.

Nuance. Voor een *gebruiker* is de intuïtie "lichte VM" niet eens zo gek: je krijgt wel degelijk een `/`, een `hostname`, een IP en een `root`. Gevaarlijk wordt ze zodra het over beveiliging gaat. De hypervisor van een VM is een echte grens; een container deelt de kernel, en één kernelfout gaat dwars door die grens heen. En op Windows is de "lichtheid" relatief: er draait wel degelijk een VM, WSL 2 — maar één enkele voor al je containers samen.

Voorbeeld.

```
time podman run --rm alpine echo hallo      # ~0,3 s, grotendeels de pull/CLI
podman run -d nginx:alpine                 # ~64 MB image, PID zichtbaar op de host via ps
```

Vraag 2 — 250 MB "zonder OS"

Antwoord. De image bevat de **userland** van een distributie: `/bin/sh`, `libc`, `coreutils`, de PostgreSQL-binaries, de configuratie. Wat **ontbreekt**, is de **kernel** — en daarmee alles wat alleen tijdens het booten bestaat: bootloader, `initrd`, kernelmodules, `systemd`, drivers, hardwarebeheer.

Waarom. Een Linux-programma praat met de kernel via systeemaanroepen (`open`, `read`, `fork`). Het hoeft dus geen kernel mee te brengen; het moet er alleen een vinden, en die van de host volstaat. De image levert enkel wat er bovenop nog ontbreekt.

Nuance. Daarom kunnen een "Alpine"- en een "Debian"-image probleemloos naast elkaar draaien op dezelfde Ubuntu onder WSL: drie userlands, één kernel. En om dezelfde reden draait een Linux-image niet op een Windows-kernel — vandaar WSL.

Voorbeeld.

```
podman run --rm alpine cat /etc/os-release | head -1 # NAME="Alpine Linux"
uname -r # 6.6.87.2-microsoft-standard-WSL2
podman run --rm alpine uname -r # STRIKT dezelfde kernel
```

Vraag 3 — Twee nginx-containers, twee verschillende `ps`

Antwoord. De `pid` -namespace. Elke container krijgt een eigen PID-tabel: zijn hoofdproces krijgt daarin nummer 1, en geen enkele externe PID is zichtbaar.

Waarom. Wie een proces niet ziet, kan er ook niets mee doen (`kill` , `/proc/<pid>`). Door het zicht weg te nemen, ontnemt de kernel de facto de mogelijkheid om langs die weg schade aan te richten. Op de host bestaan die processen gewoon, met echte PID's.

Nuance. Beveiliging "in de strikte zin" is het niet, want de grens is **niet ondoordringbaar**: het is een zichtbeperking, opgelegd door dezelfde kernel die ook de container draait. Start je een container met `--pid=host` of `--privileged` , dan is het volledige zicht terug, en een kernelfout omzeilt het mechanisme sowieso. De namespace isoleert; hij verdedigt niet. In rootless-modus legt de `user` -namespace daar wel een echte barrière van *rechten* bovenop.

Voorbeeld.

```
podman run -d --name web nginx:alpine
podman exec web ps -o pid,comm # PID 1 = nginx, dan zijn workers
ps -ef | grep -c "[n]ginx" # op de host: de processen zijn er wel degelijk
podman run --rm --pid=host alpine ps | head # isolatie weggenomen: de hele WSL
podman rm -f -t 0 web
```

Vraag 4 — Cannot connect to the Docker daemon

Antwoord. Bij je collega heeft de client zijn werk gedaan: hij toonde zijn versie, probeerde daarna de socket `/var/run/docker.sock` te openen om de **server** te bereiken, en strandde daar. Twee waarschijnlijke oorzaken: (1) de daemon draait niet, (2) zijn gebruiker heeft geen rechten op de socket. Bij jou kan deze melding niet voorkomen: Podman heeft **geen daemon** en opent geen enkele socket; elk commando doet het werk zelf, onder jouw gebruiker.

Waarom. Elk Docker-commando is in wezen een netwerkoproep naar `dockerd` . Aan het commando sleutelen heeft geen zin, want niemand heeft het al gelezen: het probleem zit een stap eerder. Podman daarentegen is een gewoon programma: als het start, werkt het.

Nuance. In twee gevallen heeft Podman *we* een server: `podman --remote` (of de variabele `CONTAINER_HOST`), dat met een `podman system service` op afstand praat, en `podman machine` op Windows/macOS, waar de Windows-client met een VM praat. Dan zou je `unable to connect to Podman socket` zien. Met Podman rechtstreeks in Ubuntu onder WSL zit je in geen van beide situaties.

Voorbeeld.

```
# Aan Docker-zijde, de diagnose:
systemctl status docker           # oorzaak 1: inactive (dead)
ls -l /var/run/docker.sock        # srw-rw---- root docker: je moet in de groep docker zitten
# Aan Podman-zijde, het bewijs dat er niets te contacteren valt:
podman version                    # één enkel blok "Client"
podman --remote version           # Error: unable to connect to Podman socket ...
```

Vraag 5 — "Een image draait niet"

Antwoord. Een image is een verzameling alleen-lezen bestanden plus metadata. Er is geen proces, geen toestand, niets om in te plannen: ze is even inert als een `.zip` met een handleiding erbij.

Waarom. Bij `podman run` voegt de engine drie dingen toe: (1) een dunne **schrijflaag** bovenop de alleen-lezen lagen, (2) een set **namespaces en cgroups**, (3) de **uitvoering** van het commando uit de metadata van de image (`ENTRYPOINT` / `CMD`), via de runtime `crun`. Het resultaat van die assemblage is de container.

Nuance. Een container *aanmaken* en hem *starten* zijn twee aparte stappen: `podman create` doet alles behalve het proces lanceren, `podman start` lanceert het. `podman run` is gewoon `create` + `start` (+ `pull` als de image nog ontbreekt).

Voorbeeld.

```
podman create --name tmp alpine sleep 30 # container aangemaakt, geen proces
podman ps -a --filter name=tmp           # STATUS: Created
podman start tmp && podman ps             # STATUS: Up -> nu draait hij
podman rm -f -t 0 tmp
```

Vraag 6 — PostgreSQL-gegevens na een `podman rm`

Antwoord. Nee, de gegevens zijn weg. En nee, de image is **niet** gewijzigd: ze is voor en na strikt identiek.

Waarom. Alles wat een container schrijft, komt (via *copy-on-write*) terecht in zijn eigen schrijflaag. `podman rm` verwijdert de container **én** die laag. De lagen van de image zijn alleen-lezen: niets van wat een container doet, kan ze aantasten. Precies daardoor vertrekken twee containers van dezelfde image gegarandeerd vanuit dezelfde toestand.

Nuance. Twee belangrijke kanttekeningen. Eén: `podman stop` vernietigt niets. Een gestopte container houdt zijn laag bij, en na `podman start` staan de gegevens er weer. Het is `rm` dat vernietigt. Twee: de officiële `postgres` -image declareert `/var/lib/postgresql/data` als `VOLUME`, waardoor de engine een **anoniem** volume aanmaakt dat de `rm` overleeft — maar zonder naam vind je het amper terug. In de praktijk beschouw je de gegevens als verloren. Benoemde volumes komen aan bod in lab 06.

Voorbeeld.

```
podman run -d --name c1 alpine sleep 600
podman exec c1 sh -c 'echo x > /merk.txt'
podman rm -f -t 0 c1
podman run --rm alpine ls /merk.txt      # No such file or directory: de image is intact
```

Vraag 7 — Tien containers, hoeveel schijf?

Antwoord. Enkele tientallen kilobytes in totaal — niet 10×210 MB. Elke container kost alleen zijn schrijfslag, die leeg begint, plus een handvol configuratiebestanden (`hostname`, `resolv.conf` ...).

Waarom. Alle containers die uit dezelfde image voortkomen, **delen** haar lagen in alleen-lezen modus. De opslagdriver `overlay` stapelt die lagen met daarbovenop één lege schrijfslag per container. Een bestand wordt pas naar die laag gekopieerd op het moment dat het gewijzigd wordt (*copy-on-write*).

Nuance. Het antwoord verandert zodra elke container veel schrijft (logs, tijdelijke bestanden): wie een bestand uit de image wijzigt, kopieert het integraal naar de laag van de container. `podman ps -s` toont beide cijfers: de eigen grootte en de "virtuele" grootte.

Voorbeeld.

```
for i in 1 2 3; do podman run -d --name t$i alpine sleep 600; done
podman ps -s --format 'table {{.Names}}\t{{.Size}}' # 11.4kB (virtual 8.72MB) elk
podman rm -f -t 0 t1 t2 t3
```

Vraag 8 — `root` in de container, `1000` op de host

Antwoord. De `user`-namespace beeldt de ID's van de container af op die van de host: UID 0 in de container is jouw UID 1000. Tegenover de kernel en de bestanden van de host heeft die "root" niet meer dan jouw rechten. Een schrijfpoging in `/etc/shadow`, gemount vanaf de host, faalt met `Permission denied` — precies alsof je het zelf probeerde.

Waarom. De kernel controleert rechten aan de hand van de **echte** identiteit (die aan hostzijde), niet de identiteit die in de namespace getoond wordt. De UID's 1 tot 65536 van de container worden afgebeeld op een gereserveerd bereik in `/etc/subuid` (`100000-165535`), dat op jouw bestanden geen enkel recht heeft.

Nuance. Binnen zijn eigen namespaces is die root wel degelijk root: hij kan pakketten installeren, rechten van imagebestanden wijzigen en luisteren op poort 80 van de container. Wat hij niet kan, is de grens oversteken. Praktisch gevolg: een bestand dat de container aanmaakt onder UID 999 (de gebruiker `postgres`) verschijnt op je host met UID 100998 — de klassieke *bind mount*-valkuil in rootless-modus (lab 06).

Voorbeeld.

```
podman top waker user,huser # root / 1000
podman unshare cat /proc/self/uid_map # 0 -> 1000 (1), 1 -> 100000 (65536)
podman run --rm -v /etc:/host alpine sh -c 'echo x >> /host/shadow' # Permission denied
```

Vraag 9 — De groep `docker` en `sudo`

Antwoord. Wie in de groep `docker` zit, mag naar `/var/run/docker.sock` schrijven — en dus eender wat laten uitvoeren door een daemon die als **root** draait: `docker run -v /:/host --privileged` levert de hele host uit. Dat is `sudo` zonder wachtwoord, zonder logging en zonder grens. Met rootless Podman is er geen root-daemon en geen socket: de gebruiker kan niets wat hij voordien niet al kon, en de regel is voorwerploos geworden.

Waarom. Auditing draait om de vraag *wie wat* gedaan heeft. Een commando dat via de socket binnenkomt, wordt uitgevoerd door `dockerd`, onder de identiteit `root`, zonder enig spoor dat naar de gebruiker leidt. `sudo docker ...` laat tenminste een regel na in `auth.log`. Rootless Podman gaat nog een stap verder: de container is een proces van de gebruiker zelf, zichtbaar en toewijsbaar in `ps`.

Nuance. Rootless Podman heeft ook een prijs: geen poorten onder 1024 zonder extra instelling, een iets trager netwerk in userspace, en sommige mounts en opties zijn uitgesloten. In productie kom je daarnaast *rootful* Podman tegen (`sudo podman`) — en dan gelden weer dezelfde voorzorgen als bij Docker.

Voorbeeld.

```
# Wat de groep docker toelaat (NIET DOEN op een gedeelde machine):
docker run --rm -v /:/host alpine cat /host/etc/shadow # leesbaar: de daemon is root
# Hetzelfde onder rootless Podman:
podman run --rm -v /:/host alpine cat /host/etc/shadow # Permission denied
```

Vraag 10 — Lange vorm, korte vorm

Antwoord.

Korte vorm	Lange vorm	Object
<code>podman ps -a</code>	<code>podman container ls -a</code>	container
<code>podman images</code>	<code>podman image ls</code>	image
<code>podman rmi nginx:alpine</code>	<code>podman image rm nginx:alpine</code>	image
<code>podman rm web</code>	<code>podman container rm web</code>	container

Waarom. `ps` en `images` stammen uit de allereerste Docker-versies (2013), toen de CLI nog geen objecten kende: `ps` bootste het gelijknamige Unix-commando na, `images` was gewoon een meervoud. De grammatica `object actie` kwam er pas in 2017 (Docker 1.13), en Podman nam ze ongewijzigd over. De korte vormen zijn blijven bestaan om niets te breken.

Nuance. Alleen de lange vorm is volledig: `podman container ls`, `podman image ls`, `podman volume ls`, `podman network ls` en `podman pod ls` volgen allemaal hetzelfde patroon, terwijl `podman ps` geen tegenhanger heeft voor volumes. Gebruik in scripts dus de lange vorm.

Voorbeeld.

```
podman container ls -a --format '{{.Names}}'
podman image ls --format '{{.Repository}}:{{.Tag}}'
```

Vraag 11 — Twee keer `uname -r`, twee hosts

Antwoord. `uname -r` is een systeemaanroep: de waarde komt van de **kernel**, nooit uit de image. Onder WSL is dat de kernel `microsoft-standard-WSL2`, die Microsoft voor de WSL-VM compileert; op een native Ubuntu-server is het de `generic`-kernel uit het Ubuntu-pakket. Een container toont altijd de kernel van de machine die hem uitvoert, welke image je ook gebruikt.

Waarom. De container is een proces van de hostkernel, en in de image zit geen kernel (vraag 2). Op Windows is die host niet Windows zelf, maar de WSL 2-VM.

Nuance. "Licht" blijft kloppen: de WSL-VM is er maar **één**, ze start één keer op en al je containers delen ze; de containers zelf blijven processen die in milliseconden starten. Wat niet meer klopt, is "helemaal geen VM". Praktische gevolgen: het beschikbare RAM is dat van WSL (`.wslconfig`), en Windows-bestanden (`/mnt/c/...`) die je in een container mount, zijn traag omdat elke toegang de grens VM ↔ Windows oversteekt. Werk dus in het Linux-bestandssysteem (`~`).

Voorbeeld.

```
uname -r # 6.6.87.2-microsoft-standard-WSL2
podman run --rm alpine uname -r # identiek
podman info --format '{{.Host.Kernel}} {{.Host.MemTotal}}' # het RAM zoals WSL het ziet
```

Vraag 12 — Oneindige lus en RAM

Antwoord. Nee: de `pid`-namespace verbergt alleen processen. Het mechanisme dat de burens beschermt, is de geheugen-**cgroup**, die je activeert met `--memory`. Zonder limiet verbruikt de container al het beschikbare RAM, en zodra de kernel niets meer overheeft, doodt de **OOM killer** een proces naar eigen keuze — niet noodzakelijk de schuldige.

Waarom. Namespaces en cgroups zijn twee onafhankelijke mechanismen: het ene isoleert het *zicht*, het andere begrenst het *verbruik*. Met `--memory=512m` sterft bij een overschrijding alleen het proces van de container zelf (`Exited (137)` , `OOMKilled: true`), en de rest van de machine merkt er niets van.

Nuance. In rootless-modus werkt `--memory` alleen als `systemd` de controller `memory` aan jouw gebruiker delegeert — op Ubuntu onder WSL is dat zo zodra `systemd=true` actief is. En voor Java geldt: een cgroup-limiet helpt alleen als de JVM ze respecteert. Sinds Java 10 leest ze de cgroup automatisch uit (`-XX:MaxRAMPercentage`), maar een handmatig te hoog ingestelde `-Xmx` gaat er alsnog overheen.

Voorbeeld.

```
podman run -d --name limiet --memory=128m --memory-swap=128m alpine sleep 600
podman stats --no-stream limiet # MEM USAGE / LIMIT: ... / 134.2MB
podman inspect --format '{{.State.OOMKilled}}' limiet # false – voorlopig
podman rm -f -t 0 limiet
```

Vraag 13 — Een image promoveren zonder ze opnieuw te bouwen

Antwoord. Twee eigenschappen: **onveranderlijkheid** (een image die je via haar digest identificeert, verandert nooit) en de **OCI-standaard** (Docker en Podman schrijven en lezen exact hetzelfde formaat). Wat in acceptatie getest is, gaat bit voor bit identiek naar productie, ongeacht de engine. Bij elke stap opnieuw bouwen breekt die garantie: twee builds van dezelfde code leveren niet noodzakelijk dezelfde image op.

Waarom. Een build hangt af van het moment waarop hij draait: `apt-get install` pakt de versie van die dag, `FROM eclipse-temurin:21-jre` volgt een verschuivende tag, Maven lost versiebereiken op. Tussen de acceptatiebuild en de productiebuild kan een afhankelijkheid veranderd zijn — en dan is de validatie in acceptatie niets meer waard.

Nuance. Promoveren doe je niet door `latest` opnieuw te taggen, maar door te verwijzen naar de **digest** (`api@sha256:...`) of naar een onveranderlijke tag (`api:1.4.2`). Lab 02 gaat daar dieper op

in. En de engine maakt nauwelijks uit: een `podman pull` van een image die met `docker push` gepusht werd, is de gewoonste zaak van de wereld.

Voorbeeld.

```
podman image inspect --format '{{.Digest}}' registry.intern/api:1.4.2
# Dezelfde digest op je eigen machine (Podman), in acceptatie (Docker) en in productie (rootful Podman).
```

Vraag 14 — `alias docker=podman`

Antwoord. Klopt zonder voorbehoud voor: (1) de volledige image-cyclus — `build`, `pull`, `push`, `tag`, `images`, `history`, `inspect`, de Dockerfiles; (2) de levenscyclus van containers — `run`, `ps`, `logs`, `exec`, `stop`, `rm`, opties inbegrepen. Klopt niet, of loopt anders: (a) **geen daemon** — geen `docker.sock`, `--restart=always` overleeft geen herstart zonder `systemd`, `podman rm -f` wacht 10 s; (b) **rootless** — geen poorten onder 1024 zonder extra instelling, geen IP-adressen met `pasta`, verschoven UID's op *bind mount*-bestanden, `--memory` alleen met cgroup-delegatie.

Waarom. Podman heeft het *oppervlak* van Docker overgenomen (de CLI, het formaat), maar niet de *architectuur*. Alles wat alleen van dat oppervlak afhangt, is identiek; alles wat raakt aan "wie voert uit, met welke rechten, onder wiens toezicht" loopt uiteen.

Nuance. De verschillen zijn geen tekortkomingen: elk ervan is de keerzijde van een veiligheidskeuze. En Docker Compose werkt ook met Podman (`podman compose`, lab 09), mits enkele regels configuratie.

Voorbeeld.

```
alias docker=podman
docker run -d --name web -p 8080:80 nginx:alpine      # identiek
docker run -d --name w80 -p 80:80 nginx:alpine      # Error: pasta failed ... Listen failed for
HOST TCP port */80: Permission denied
podman rm -f -t 0 web
```